



IS231: Web Technology HTML Document Object Model



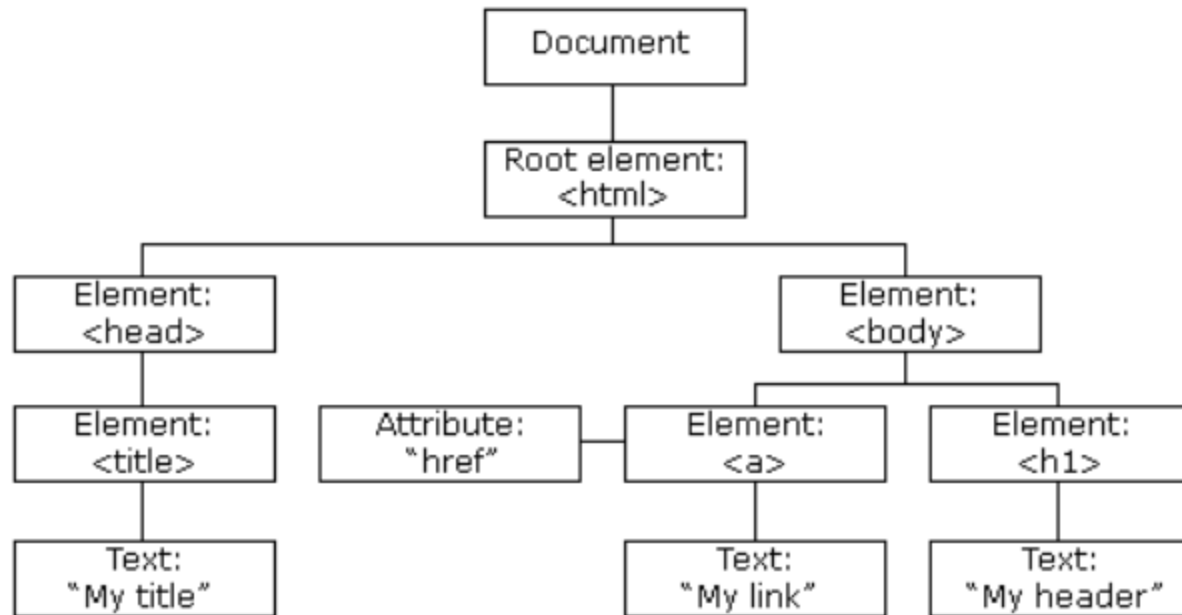
By: Neamat El Tazi

References

- **Some Figures and Slides References**
 - **W3schools.com**

HTML DOM Tree

The HTML DOM Tree of Objects



With the HTML DOM, all nodes in the node tree can be accessed by JavaScript.
New nodes can be created, and all nodes can be modified or deleted.

Node Relationships

- ▶ The nodes in the node tree have a hierarchical relationship to each other.
- ▶ The terms parent, child, and sibling are used to describe the relationships.
 - In a node tree, the top node is called the root (or root node)
 - Every node has exactly one parent, except the root (which has no parent)
 - A node can have a number of children
 - Siblings (brothers or sisters) are nodes with the same parent

HTML Document

```
<html>
```

```
<head>
```

```
<title>DOM Tutorial</title>
```

```
</head>
```

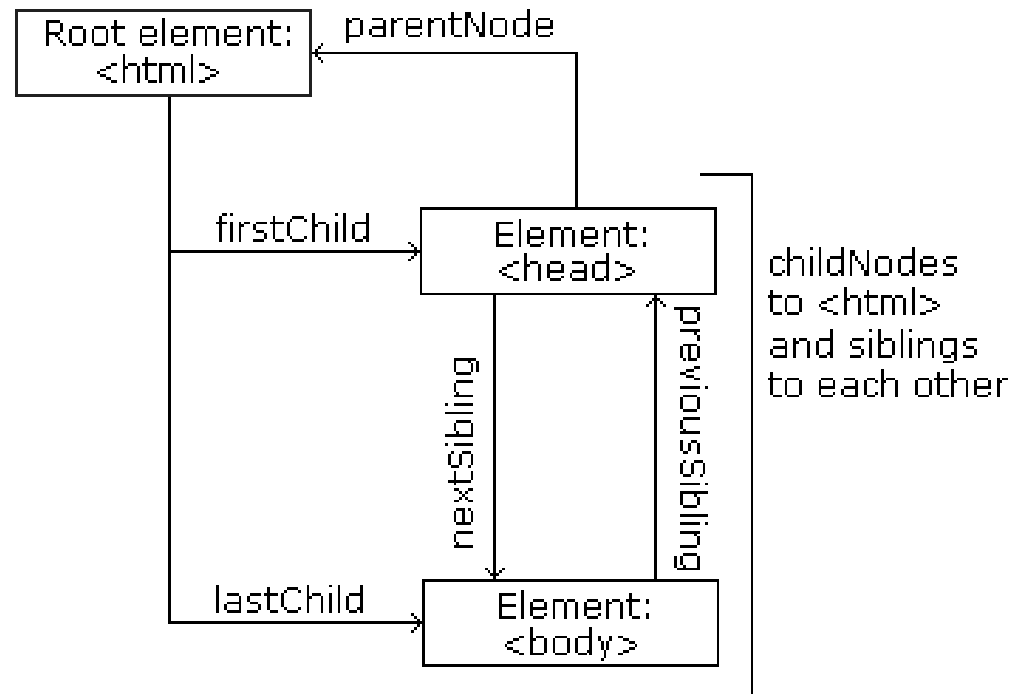
```
<body>
```

```
<h1>DOM Lesson one</h1>
```

```
<p>Hello world!</p>
```

```
</body>
```

```
</html>
```



HTML Document

```
<html>
```

```
  <head>
```

```
    <title>DOM Tutorial</title>
```

```
  </head>
```

```
  <body>
```

```
    <h1>DOM Lesson one</h1>
```

```
    <p>Hello world!</p>
```

```
  </body>
```

```
</html>
```

From the HTML you can read:

- `<html>` is the root node
- `<html>` has no parents
- `<html>` is the parent of `<head>` and `<body>`
- `<head>` is the first child of `<html>`
- `<body>` is the last child of `<html>`
- `<head>` has one child: `<title>`
- `<title>` has one child (a text node): "DOM Tutorial"
- `<body>` has two children: `<h1>` and `<p>`
- `<h1>` has one child: "DOM Lesson one"
- `<p>` has one child: "Hello world!"
- `<h1>` and `<p>` are siblings

HTML Document Object Model (DOM)

▶ Tree representation

▶ HTML document

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML//EN">
<html> <head>
<title></title>
</head>

<body>
<h1></h1>

|

<hr>
<address></address>
<!-- hhmts start --><!-- hhmts end -->
</body> </html>
```

◀ ▶ | test.html > DOM Tree > E html > E body > E h1

```
<!DOCTYPE html PUBLIC "-//IETF//DTD HTML//EN">
▼ <html>
  ▶ <head>...</head>
  ▼ <body>
    <h1></h1>
    <hr>
    <address></address>
    <!-- hhmts start -->
    "Last modified: Mon Sep 1 20:36:18 MST 2014 "
    <!-- hhmts end -->
  </body>
</html>
```

html: document.documentElement

body: document.body

HTML DOM Tree

What is the HTML DOM?

The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

In other words: **The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**

Navigating Between Nodes

- ▶ You can use the following node properties to navigate between nodes with JavaScript:
 - ▶ parentNode
 - ▶ childNodes[nodenumber]
 - ▶ firstChild
 - ▶ lastChild
 - ▶ nextSibling
 - ▶ previousSibling

Child Nodes and Node Values

▶ `<p id="demo">DOM Tutorial</p>`

The element node `<p>` (in the example above) does not contain text.

It contains a text node with the value "DOM Tutorial".

The value of the text node can be accessed by the node's `innerHTML` property:

```
t = document.getElementById("demo").innerHTML;
```

Document Object Model (DOM) - Navigating

Properties for traversing the DOM:

- ▶ **e.childNodes**
 - ▶ children of element e as an array, e.g. `childNodes[0]`
- ▶ **e.children**
 - ▶ element nodes only (excludes text nodes)
- ▶ **e.firstChild**
- ▶ **e.lastChild**
- ▶ **e.parentNode**
- ▶ **e.nextSibling**
- ▶ **e.previousSibling**

Object properties:

- ▶ **e.nodeType**
 - ▶ 1 = element, 3 = text, ...attr
- ▶ **e.nodeName**
 - ▶ uppercase
- ▶ **e.innerHTML**
 - ▶ for element nodes
 - ▶ value is html as text
 - ▶ writable
- ▶ **e.nodeValue**
 - ▶ for text nodes
(null: for element nodes)
 - ▶ writable



Tricky Question

- ▶ What does *firstChild* return for `<ul>\n <li>...`?

```
ul.firstChild.nodeType;    // 3 = TEXT (the newline character!)  
ul.firstChild.nodeName;   // "#text"
```

 Use these instead:

```
ul.firstElementChild.nodeType; // 1 = ELEMENT ✓  
ul.firstElementChild.nodeName; // "LI" ✓
```

// Node type reference:

// 1 = Element (<p>, <div>, etc.)

// 3 = Text (text content, whitespace)

// 8 = Comment (<!-- ... -->)

Finding HTML Elements (Methods)

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

Document Object Model (DOM) – Finding Elements

Locating an element:

- ▶ `document.getElementById(id)`
 - ▶ useful if you have named the document element using the `id='Name'` property
- ▶ `document.getElementsByTagName(tag)`
 - ▶ all document elements of type tag
 - ▶ returns an array
- ▶ `e.getElementsByTagName(tag)`
 - ▶ all elements of type tag under element e
 - ▶ returns an array
- ▶ `document.getElementsByName(name)`
 - ▶ useful for elements that support `name='Name'`
- ▶ `(document|e).getElementsByClassName(class)`
- ▶ `(document|e).querySelector(query)`
 - ▶ example query `'BODY > UL > LI'`
 - ▶ `'>'` means immediately dominates
 - ▶ returns first matching element
- ▶ `(document|e).querySelectorAll(query)`
 - ▶ returns an array



5 Ways to Find Elements - Examples

// ① By ID — fastest, returns *ONE* element

```
document.getElementById('title');    // <h1>
```

// ② By class name — returns *live HTMLCollection*

```
document.getElementsByClassName('note'); // [<p>, <p>]
```

// ③ By tag name

```
document.getElementsByTagName('li');    // [<li>Mango, <li>Guava]
```

// ④ *querySelector* — CSS selector, returns *FIRST* match

```
document.querySelector('input[type="email"]'); // the <input>  
document.querySelector('#fruits > li');    // Mango (direct child)
```

// ⑤ *querySelectorAll* — CSS selector, returns *static NodeList*

```
document.querySelectorAll('.note');    // [<p>, <p>]
```

// *Scoped search* — search *INSIDE* a specific element only

```
const ul = document.getElementById('fruits');  
ul.querySelectorAll('li'); // only li's inside #fruits
```

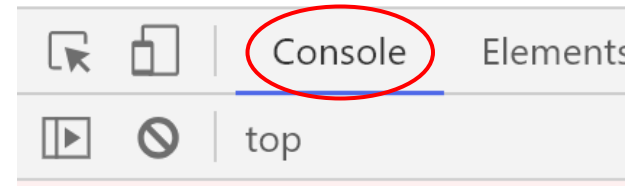
```
<h1 id="title">DOM</h1>  
<p class="note">First note</p>  
<p class="note">Second note</p>  
<input type="email"  
  name="email">  
<ul id="fruits">  
  <li>Mango</li>  
  <li>Guava</li>  
</ul>
```

Demo

← → ↻ ⓘ 127.0.0.1:90/list.html

Apps The Open Source... Performance and... Free Learning | PA... Implemetation of...

- This pages shows us how lists work
- We need to encode certain characters to show them in HTML
- We should learn more about Paragraphs
- We should learn about Images



```
> document.getElementsByTagName("ul")
```

```
<> ▶ HTMLCollection [ul]
```

```
▶ childNodes: NodeList(9) [text, li, text, li, text, li, text, li, text]
▶ children: HTMLCollection(4) [li, li, li, li]
```


Demo- Continue

```
▶ childNodes: NodeList []  
  data: "This pages shows us how lists work"  
  firstChild: null
```

```
> x =document.body.getElementsByTagName("ul")
```

```
◀ ▶HTMLCollection [ul]
```

```
> x.children
```

```
◀ undefined
```

```
> x[0].children
```

```
◀ ▶HTMLCollection(4) [li, li, li, li]
```

Demo- Continue

```
x[0].innerHTML
```

```
"
```

```
<li><p>This pages shows us how lists work</p></li>  
<li><p>We need to encode<a href="page1.html">certain c  
<li><p>We should learn more about<a href="page2.html">  
<li><p>We should learn about<a href="page3.html"> Image  
</p></li>
```

```
"
```

Demo- Continue

```
x[0].innerHTML
```

```
"
```

```
<li><p>This pages shows us how lists work</p></li>  
<li><p>We need to encode<a href="page1.html">certain c  
<li><p>We should learn more about<a href="page2.html">  
<li><p>We should learn about<a href="page3.html"> Image  
</p></li>
```

```
"
```

Demo- Continue

```
> items = x[0].children
```

```
< ▶ HTMLCollection(4) [li, li, li, li]
```

```
> items[0].innerHTML
```

```
< "<p>This pages shows us how lists work</p>"
```

Changing HTML Elements

Property	Description
<i>element.innerHTML = new html content</i>	Change the inner HTML of an element
<i>element.attribute = new value</i>	Change the attribute value of an HTML element
<i>element.style.property = new style</i>	Change the style of an HTML element
Method	Description
<i>element.setAttribute(attribute, value)</i>	Change the attribute value of an HTML element

Changing HTML Elements - Examples

```
const h1 = document.getElementById('title');
```

```
// Change text content
```

```
h1.textContent = 'New Title';
```

```
// Change with HTML inside (renders tags)
```

```
h1.innerHTML = 'Hello <em>World</em>';
```

```
// Change styling
```

```
h1.style.color = 'tomato';
```

```
h1.style.fontSize = '3rem';
```

```
// Change an attribute
```

```
const link = document.querySelector('a');
```

```
link.setAttribute('href', 'https://example.com');
```

```
link.getAttribute('href'); // read it back
```

```
// Change class
```

```
h1.className = 'highlight bold';
```

```
<h1 id="title">DOM</h1>
  <p class="note">First
note</p>
  <p class="note">Second
note</p>
  <input type="email"
name="email">
  <a href="www.google.com">My
Link goes to Google</a>
  <ul id="fruits">
    <li>Mango</li>
    <li>GuaVa</li>
  </ul>
```

InnerText Vs. InnerHTML

◆ 1 innerText

- Returns or sets the **visible text only**
- Ignores hidden elements (`display: none`)
- Does NOT interpret HTML tags
- Affected by CSS styling

Example

«» HTML

```
<div id="demo">
  Hello <b>World</b>
</div>
```

«» JavaScript

```
document.getElementById("demo").innerText;
```

👉 Output:

«» Code

Hello World

◆ 2 innerHTML

- Returns or sets the **HTML content**
- Includes tags
- Interprets HTML when setting content

«» JavaScript

```
document.getElementById("demo").innerHTML;
```

👉 Output:

«» Code

Hello World

Tags are preserved.

InnerText Vs. InnerHTML



Security Warning (Very Important)

Using `innerHTML` with user input can cause:

! XSS (Cross-Site Scripting) attacks

Example:

```
</> JavaScript
```

```
element.innerHTML = userInput;
```

If `userInput` contains:

```
</> HTML
```

```
<script>alert("Hacked")</script>
```

It will execute.



That's why `innerText` is safer when inserting plain text.

Adding and Deleting Elements

Method	Description
<code>document.createElement(<i>element</i>)</code>	Create an HTML element
<code>document.removeChild(<i>element</i>)</code>	Remove an HTML element
<code>document.appendChild(<i>element</i>)</code>	Add an HTML element
<code>document.replaceChild(<i>new</i>, <i>old</i>)</code>	Replace an HTML element
<code>document.write(<i>text</i>)</code>	Write into the HTML output stream

Document Object Model (DOM) - Updating

New content:

- ▶ `document.createElement(tag)`
 - ▶ `tag = 'div', 'p' etc.`
 - ▶ creates new DOM element
- ▶ `document.createTextNode(text)`
 - ▶ creates new DOM element of type text

ADVANCED:

For non-HTML elements:

- ▶ `document.createElementNS(NS,tag)`
 - ▶ NS = Namespace URL identifier
 - ▶ e.g. <http://www.w3.org/2000/svg> and tag “rect” (rectangle)

Place new_el:

- ▶ `e.appendChild(new_el)`
 - ▶ `new_el` is inserted as last child of `el`
- ▶ `e.insertBefore(new_el,next_el)`
 - ▶ `new_el` inserted as previous sibling of `next_el`
 - ▶ `el` is common parent
- ▶ `e.removeChild(child_el)`
 - ▶ `child_el` is deleted
 - ▶ `el` is parent
- ▶ `e.replaceChild(new_el,child_el)`
 - ▶ `new_el` replaces `child_el`
 - ▶ `el` is parent

Adding or Deleting Elements - Examples

// ① Create a new element

```
const li = document.createElement('li');  
li.textContent = 'Buy groceries';
```

// ② Add it to the list

```
const ul = document.getElementById('to-do-list');  
ul.appendChild(li);           // add at END  
// ul.prepend(li);           // add at START (modern)
```

// ③ Insert BEFORE a specific item

```
const urgent = document.createElement('li');  
urgent.textContent = '⚡ Urgent task';  
ul.insertBefore(urgent, ul.firstChild);
```

// ④ Remove an element

```
ul.removeChild(li);           // old way  
// li.remove();              // modern — same result
```

// ⑤ Replace an element

```
const done = document.createElement('li');  
done.textContent = '✅ Done';  
ul.replaceChild(done, urgent);
```

```
<h1 id="title">DOM</h1>  
<p class="note">First note</p>  
<p class="note">Second note</p>  
<ul id="to-do-list">  
  <li>Study HTML</li>  
  <li>Study DOM</li>  
</ul>
```

Adding Events Handlers

Method	Description
<code>document.getElementById(id).onclick = function() {code}</code>	Adding event handler code to an onclick event

// Common events to know:

`btn.addEventListener('click', handler);` *// mouse click*

`btn.addEventListener('dblclick', handler);` *// double click*

`btn.addEventListener('mouseenter', handler);` *// hover in*

`document.addEventListener('keydown', handler);` *// key press*

`document.addEventListener('DOMContentLoaded', handler);` *// page ready*

Not the best practice).. Why?

```
document.getElementById("myid").onclick = function() { alert('hello'); };
```

Modern DOM APIs (ES6+)

- ▶ **Cleaner Element Traversal:**

- ▶ `element.closest(selector)` — walk UP the tree to find matching ancestor
- ▶ `element.matches(selector)` — check if element fits a selector (true/false)

- ▶ **Modern insertion (shorter than `appendChild` / `removeChild`):**

- ▶ `parent.append(el)` `parent.prepend(el)` `el.remove()` `el.replaceWith(newEl)`

Modern DOM APIs (ES6+)

▶ **classList API (modern alternative to className)**

```
const box = document.getElementById('box');

// Old way - fragile string manipulation:
box.className = box.className + ' active'; // ✗

// Modern classList API:
box.classList.add('active');           // add
box.classList.remove('active');        // remove
box.classList.toggle('active');        // add if missing, remove if present
box.classList.contains('active');      // true / false
box.classList.replace('old', 'new');   // swap a class
```

Toggle: Add if missing, remove if available

Modern DOM APIs (ES6+)

► Dataset API

❏ HTML

```
<button id="btn"  
  data-user-id="42"  
  data-role="admin">  
  Click Me  
</button>
```

◆ Accessing Data in JavaScript

❏ JavaScript

```
const btn = document.getElementById("btn");  
  
console.log(btn.dataset.userId); // "42"  
console.log(btn.dataset.role);   // "admin"
```

Modern Event Handling

- ▶ **Use `addEventListener` instead of `onclick` (supports multiple handlers):**
 - ▶ `btn.addEventListener('click', handler)` vs `btn.onclick = handler`
- ▶ **Event Object — passed automatically to your handler:**
 - ▶ `event.target` — the element that was clicked
 - ▶ `event.preventDefault()` — stop default browser behavior (e.g., form submit, link follow)
 - ▶ `event.stopPropagation()` — stop the event from bubbling up to parent elements
- ▶ **Event Bubbling — events travel from child UP to parent:**
 - ▶ Clicking a `<button>` inside a `<div>` triggers both the button and div click handlers
- ▶ **Event Delegation — attach ONE listener to a parent for many children:**
 - ▶ `document.querySelector('#list').addEventListener('click', e => {`
 - ▶ `if (e.target.tagName === 'LI') doSomething(e.target); });`
 - ▶ More efficient than adding a listener to every list item individually

DOM Performance Tips

- ▶ **Problem: Every DOM read/write can trigger a browser reflow (expensive!):**
 - ▶ Avoid mixing reads and writes in a loop — batch them separately
 - ▶ Check the interactive demo!
- ▶ **Use DocumentFragment for bulk insertions:**
 - ▶ `const frag = document.createDocumentFragment();`
 - ▶ `items.forEach(item => frag.appendChild(createRow(item)));`
 - ▶ `table.appendChild(frag);` // Only ONE reflow instead of N reflows
- ▶ **Use textContent over innerHTML when no HTML is needed:**
 - ▶ `el.textContent = userInput` (safe — no XSS risk)
 - ▶ `el.innerHTML = userInput` (dangerous if input is untrusted — XSS attack vector (Cross site scripting risk))

AI in DOM Development & Testing

▶ **AI for Development:**

- ▶ Generate DOM boilerplate — describe what you want, AI writes the `querySelector` / event handler
- ▶ Explain DOM errors — paste a console error, AI diagnoses and fixes it
- ▶ GitHub Copilot & Claude autocomplete DOM manipulation as you type
- ▶ Ask AI to visualize a DOM tree: 'What does the DOM look like for this HTML snippet?'

▶ **AI for Testing:**

- ▶ AI-powered test generation — describe user interaction, AI writes Playwright/Cypress test code
- ▶ TestRigor / Healenium — AI that auto-heals broken selectors when DOM changes
- ▶ BrowserStack AI — analyze test failures, suggest fixes from screenshots

AI in DOM Development & Testing

▶ **Prompting Tips for DOM tasks:**

- ▶ 'Write a JS function that adds a delete button to each in #todo-list'
- ▶ 'Find the bug in this addEventListener code: [paste code]'
- ▶ 'Generate 5 unit tests for my DOM manipulation function using Jest'

NEXT

CSS